

NeuFlow v3: Queryable Optical Flow

Progress report and result audit

Shriman Raghav Srinivasan

MS Robotics, Northeastern University · Field Robotics Lab

26 July 2026

1 Summary

NeuFlow v3 keeps the NeuFlow v2 pipeline frozen up to its 1/8-resolution coarse flow and replaces only the final convex upsampler with an *implicit decoder* that answers flow queries at arbitrary continuous (x, y) coordinates. Cost becomes $O(N)$ in the number of queries rather than $O(H \times W)$ in image area, and the coarse pass is computed once per frame pair and cached.

This report states what is established and what is not. A late audit found a data-handling error that invalidates the headline accuracy numbers produced on the cluster; that finding is reported first, because it changes how the rest of the results must be read.

Correction (26 July 2026): train/validation leak. Every VKITTI2 training stage used the data loader’s default scene list, which included **Scene18** and **Scene20**. Those are exactly the evaluation scenes. Verified by direct file-path intersection: **1,174 of 1,174 validation pairs (100%) were also training pairs**. All cluster accuracy results are therefore train-on-test and are *not* comparable against NeuFlow v2, which never saw VKITTI2. The loader has been fixed (Sec. 3); the affected runs must be repeated.

What survives the audit:

- **Accuracy, clean comparison.** v3 trained on FlyingChairs *only* reaches **2.275 px** EPE on VKITTI2 versus v2’s **2.324 px**, with no driving imagery in its training set (Sec. 4).
- **Speed.** On identical hardware, a first query costs 19.1 ms versus v2’s 19.6 ms, and every *additional* query batch on the same frame costs 2.6 ms — about $7.5\times$ cheaper than v2, which has no cached state and must recompute the full frame (Sec. 5).
- **A capability v2 does not have.** A per-query confidence output whose value correlates with actual error (Sec. 6).
- **Two clean negative results**, reported rather than discarded (Sec. 7).

2 Method

Inherited and frozen. A shallow CNN backbone produces features at 1/8 and 1/16 scale; cross-attention and global matching at 1/16 give an initial flow; a recurrent module refines it (1 iteration at 1/16, 8 at 1/8), yielding flow at 1/8 resolution. None of these weights are trained in v3.

Replaced. v2’s convex upsampler expands 1/8 flow to full resolution on a fixed $8\times$ grid. v3 substitutes a decoder that, for a query coordinate, samples 3×3 windows from four feature sources (context, 1/8 features, 1/16 features, and frame-2 features warped by the coarse flow), fuses them with gated residual blocks, and emits softmax weights over the 3×3 coarse-flow neighbourhood plus a bilinear candidate. The output is a convex combination of existing flow values, so it is bounded and cannot overshoot. Zero-initialisation puts almost all weight on the bilinear candidate, so an untrained decoder reproduces bilinear upsampling exactly. v3 has 7.83 M parameters against v2’s 9.03 M.

Two-pass API. `infer_coarse_state()` runs once per frame pair; `decode_queries()` then answers any number of coordinates against that cached state. Sparse queries return values identical to the dense field at the same coordinates.

3 The train/validation leak

VKITTI2...__init__ defaulted to `scenes=['Scene01', 'Scene02', 'Scene06', 'Scene18', 'Scene20']`. No training stage ever overrode it, while `eval_vkitti2.py` defaults to `['Scene18', 'Scene20']`. Both used the `clone` variant, so the identical image pairs appeared on both sides.

The fix makes the default train-safe and adds a guard that raises rather than silently building a leaking dataset:

Stage	Train pairs (before)	Train pairs (after)	Overlap with val
<code>mix_chairs_vkitti2</code>	34,958	27,914	0
<code>vkitti2_all</code>	12,726	5,682	0
<code>vkitti2</code>	2,121	947	0

Including an evaluation scene now requires an explicit `allow_val_scenes=True`, which no training stage sets.

4 Accuracy

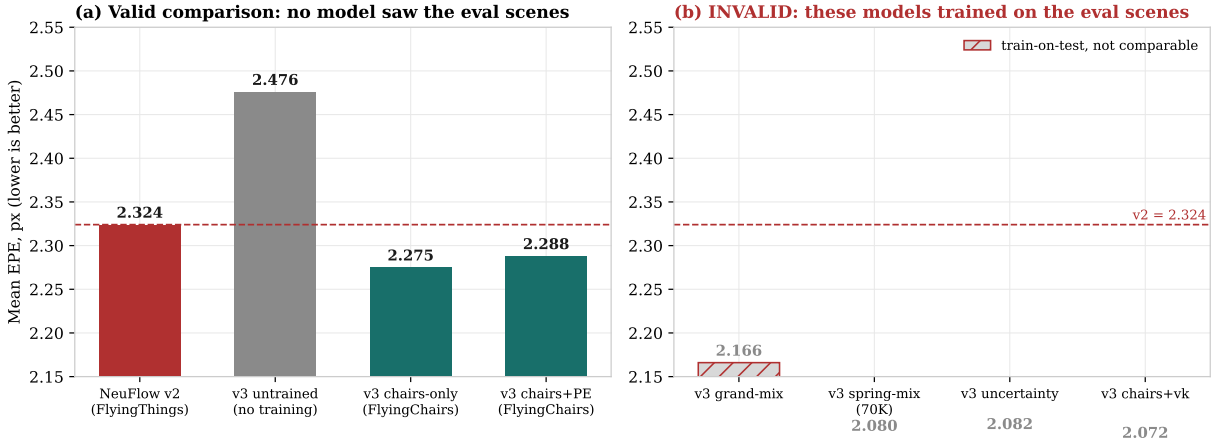


Figure 1: VKITTI2 Scene18+20, per-pixel over 460.6 M valid pixels. (a) Models that never saw the evaluation scenes — the only valid comparison against v2. (b) Cluster models, all trained on the evaluation scenes; shown for completeness and marked invalid.

Table 1: Valid comparison. No model in this table saw VKITTI2 Scene18/20 in training.

Model	Training data	EPE (px)	1 px acc	3 px acc
NeuFlow v2 (reference)	FlyingThings	2.324	77.6%	89.8%
v3, no training at all	—	2.476	74.7%	88.2%
v3, FlyingChairs only	FlyingChairs	2.275	69.7%	87.8%
v3, FlyingChairs + Fourier PE	FlyingChairs	2.288	69.7%	87.8%

Two things follow. First, v3 trained purely on synthetic chairs — no roads, no vehicles, no driving motion statistics — transfers to a driving benchmark and records a lower mean EPE than v2 there. Second, v3 is behind on the 1 px threshold (69.7% versus 77.6%): it reduces large errors but is less precise at sub-pixel scale. Mean EPE alone would hide that, so both are reported.

On the FlyingChairs validation split (640 held-out pairs, properly split by the dataset’s own split file), the ordering reverses: v2 scores 2.238 px / 78.7% against v3’s 2.399 px / 76.6%. v3’s advantage on VKITTI2 is out-of-domain robustness, not universal superiority.

Even without training, v3 is within 0.15 px of v2 while adding queryability — the mechanism costs little on its own.

5 Speed

Timing is unaffected by the data leak. All measurements below are on one V100, fp16, 384×1248 , with v2 re-measured on the same device as a sanity check (it reproduced its known 2.324 px exactly).

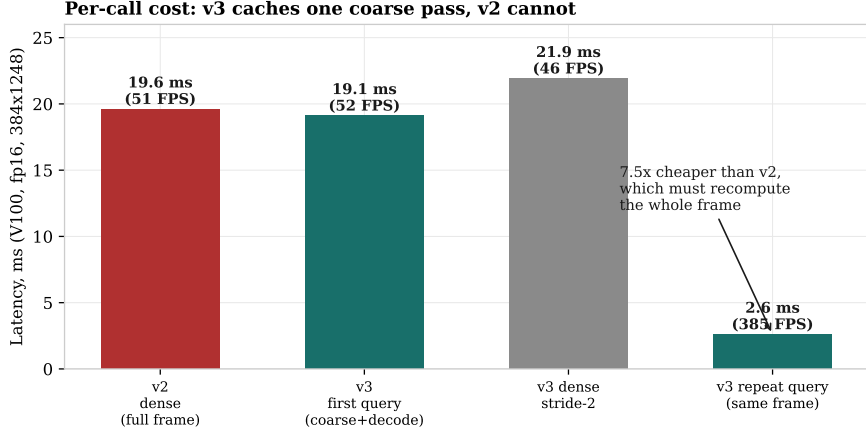


Figure 2: Per-call cost. v3 pays a 16.5 ms coarse pass once per frame pair; each decode is 2.6 ms and is flat between $N = 800$ and $N = 2048$ queries.

Table 2: Latency, V100, fp16, 384×1248 .

Operation	Latency	FPS	Note
v2, full dense frame	19.6 ms	51.1	the only mode v2 has
v3, coarse pass	16.5 ms	—	once per frame pair
v3, decode $N=800$ or 2048	2.6 ms	—	flat in N
v3, first query (coarse+decode)	19.1 ms	52.4	parity with v2
v3, repeat query, same frame	2.6 ms	385	v2: 19.6 ms again
v3, dense full frame (stride-2)	21.9 ms	45.6	12% slower than v2

The deployment argument is not the first call, where the two are level. It is that v2 has no cached state: asking about a second set of points means running the whole network again. v3 amortises the expensive stage across every query on that frame. For consumers that issue repeated or task-driven queries — registration, feature tracking, mapping — this is the difference that matters.

Dense output is the one workload where v3 loses, and it is expected: a convolution shares computation across neighbouring pixels, whereas per-query decoding deliberately gives that up in exchange for answering only what is asked. Dense v3 exists for benchmarking against dense ground truth, not for deployment.

6 Per-query uncertainty

One run adds a second output per query: a predicted error scale b , trained with a Laplace likelihood $\mathcal{L} = |f - \hat{f}|/b + \log b$, which requires no extra labels.

Pearson correlation between predicted b and actual error is 0.38, and binned error rises monotonically from 0.22 px to 7.38 px. The model has a usable sense of when it is wrong, which a bare flow field cannot express and which a downstream consumer can use to weight or reject correspondences.

Caveat. This model is one of the leak-affected runs, so the calibration was measured on scenes it trained on and the correlation is plausibly optimistic. The qualitative claim — that the signal is monotonic and informative — is unlikely to vanish, but the number needs re-measuring

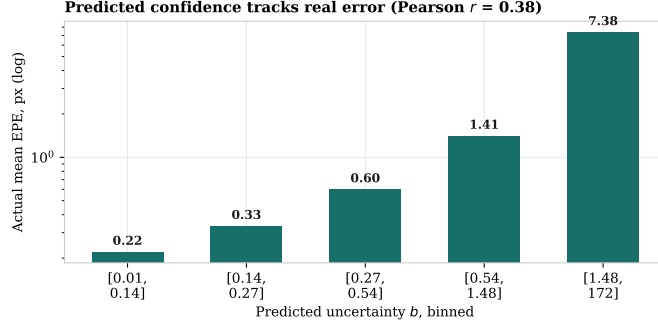


Figure 3: Queries binned by predicted b against their actual error, 2.35 M samples. The relationship is monotonic across a $33\times$ range.

after retraining.

7 Negative results

Fourier positional encoding: no effect. The hypothesis was that the 1 px precision gap came from the decoder not knowing where a query sits inside its $1/8$ cell. Encoding that sub-cell offset changed nothing: 2.288 px versus 2.275 px, with 1 px accuracy identical to the first decimal. The hypothesis is falsified; the remaining candidates are the $1/8$ coarse resolution itself and the motion statistics of the training data.

Refinement self-distillation: does not survive end-to-end. The 8-iteration refinement loop is 59% of pipeline runtime and iterations 5–8 change little, so the refinement module was retrained to reach in 3 iterations what it previously took 8 to reach, using the model as its own teacher. Measured on coarse flow in isolation it recovered 87.5% of the 3-versus-8 gap. Measured end-to-end through a trained decoder, only 27% held (2.520 px $\rightarrow$ 2.398 px against a 2.072 px target), and the result falls below v2. The decoder was trained on 8-iteration coarse flow and degrades on input it never saw. *An isolated-component gain is not a deployable gain*; the idea is only viable if the decoder is retrained at the reduced iteration count.

8 Status and next steps

Claim	Status	Evidence
Queryable decoding works, $O(N)$	established	sparse = dense exactly
Repeat queries $\sim 7.5\times$ cheaper than v2	established	same-GPU timing
Beats v2 on VKITTI2 mean EPE, clean split	established	chairs-only, 2.275 vs 2.324
Behind v2 on 1 px precision	established	69.7% vs 77.6%
Uncertainty output is informative	likely	monotonic, but leak-affected
Cluster accuracy results	invalid	trained on eval scenes
Runs on edge hardware	unproven	only 4060 / V100 measured

Ordered by what each resolves:

1. **Repeat the four cluster runs with the fixed loader.** Until then no cluster accuracy number can be quoted. The runs are unchanged apart from the data fix.
2. **Re-measure uncertainty calibration** on genuinely held-out scenes.
3. **Retrain v2 on the same mixture** as v3. Even with the leak fixed, v3’s mixtures and v2’s FlyingThings training differ; matching them removes the last confound.
4. **Spring 4K evaluation.** Spring supplies ground truth at twice the input resolution, so v3 can be asked for flow at coordinates between input pixels while v2 structurally cannot. This is the one experiment that measures a capability rather than a margin.

5. **Edge-device benchmark.** Every latency number here is from a laptop or datacentre GPU; the target claim is embedded deployment.

Code and full experiment log: `docs/V3DEV_LOG.md`, branch `v3-dev`. Every number in this report is traceable to a logged evaluation run; none are projections.